SOFTWARE CONSTRUCTION & DEVELOPMENT

Assignment 02

ExpenseMate

Software Evolution: Migration, Refactoring & Rewriting

Final Report with Supporting Reports

Submitted by

Roll No.	Name	Section
24K-3010	Erfa Kashif	SE-5B
24K-3037	Daneya Shafiq	SE-5B

Project: ExpenseMate, Personal Expense Manager

Repository branch: assignment-02 · Baseline: tag assignment-01-baseline

November 2026

Contents

1. Introduction 3
2. Migration (details: Appendix A) 3
3. Refactoring (details: Appendix B and D) 3
4. Rewriting (details: Appendix C) 4
5. Testing and regression 4
6. Maintainability metrics (details: Appendix E) 6
7. Team contributions 6
8. Challenges, trade-offs and lessons learned 6
Appendix A: Migration Report 8
1. Before state (Assignment 01, SQLite) 8
2. Target state (Assignment 02, PostgreSQL 16) 8
3. Reason for migration 8
4. Implementation procedure 9
5. Data preservation 10
6. Risks and mitigation 10
7. Verification evidence (actual run, docs/evidence/migration_verification.md) 10
Appendix B: Refactoring Report 11
1. Old architecture 11
2. New architecture: Controller → Service → Repository → Database 11
3. The two refactored features 11
4. Dependency direction and injection 12
5. Code-quality improvements (with evidence) 12
6. Behaviour intentionally changed (documented, all tested) 12
Appendix C: Rewriting Report 14
What existed 14
Rewrite 1: Budget calculation module 14
Rewrite 2: Expense management module 15
Result 16
Appendix D: Architecture Before and After 17
Before (Assignment 01, tag assignment-01-baseline) 17
After (Assignment 02, branch assignment-02) 17
Feature flows 18
Folder structure (actual) 18
Appendix E: Maintainability Comparison 20
How each metric was measured 20
Results 20
Interpretation (honest) 21
Appendix F: Change Log 22
Appendix G: Migration Verification Output 24
Appendix H: Defect Probes (Before vs After) 25
Before (assignment-01-baseline) 25
After (assignment-02) 25

1. Introduction

ExpenseMate is a personal expense manager. A user registers (JWT login), records income and expenses in any of 20 currencies, sets monthly budgets per category, receives warning (80%) and critical (95%) alerts, sees category-wise and monthly-trend charts (Chart.js), converts currencies, and imports or exports CSV files. Assignment 01 delivered this with Flask, SQLite and an HTML/CSS/JS frontend.

Assignment 02 evolves the *same* application in three ways. It migrates the database to PostgreSQL, refactors the backend into a Controller → Service → Repository → Database architecture, and rewrites the two worst modules. The frontend was not changed at all: the API contract was kept, and this was checked by the original tests and a browser run.

2. Migration (details: Appendix A)

SQLite was replaced by PostgreSQL 16 using the psycopg 3 driver. Credentials come from `DATABASE_URL` / `TEST_DATABASE_URL` (a `.env` file that is git-ignored; `.env.example` is provided). The schema is versioned SQL (`database/migrations/001_initial_schema.sql`) applied automatically and idempotently at start-up. Type changes include `REAL` → `NUMERIC(14,2)` for exact money, `TEXT` → `DATE`/`TIMESTAMP`, identity keys, all foreign keys and checks, plus a month check and a `(user_id, date)` index.

`migration/migrate_sqlite_to_postgres.py` is repeatable and safe. It opens SQLite read-only (checksum unchanged), copies every row with its original id inside one transaction, resets the sequences, and verifies the copy (counts, row-by-row comparison, money totals, orphan FKs, next ids). It commits only if every check passes, refuses to overwrite existing data without `--replace`, and rolls back on any error. On the real data (2 users, 11 categories, 12 transactions, 3 budgets) **16/16 checks passed**.

3. Refactoring (details: Appendix B and D)

The old `server.py` → `Logic` → `Database` modules only *claimed* to be layers. Route handlers called the database directly (4 places) and called private logic methods (3 places), and `Logic` had 26 methods covering about 7 responsibilities. The new backend has:

- **controllers/**: four Flask blueprints that only parse input, perform basic checks, call one service and return JSON, plus central error handlers (400/401/404/409/500/503);
- **services/**: business rules (expense, budget, analytics, user, currency), with no SQL and no Flask;
- **repositories/**: all SQL, parameterised, one class per table;
- **database/**: configuration, connections, a shared-connection `transaction()` and schema migrations.

`server.py` is the composition root that injects every dependency. Feature 1 (**Add Expense**) flows `ExpenseController → ExpenseService → ExpenseRepository → PostgreSQL`. Feature 2 (**Monthly Budget Summary**) flows `BudgetController → BudgetService → BudgetRepository → PostgreSQL`. The new boundaries fixed real bugs: invalid input returned HTTP 500, and any logged-in user could delete any other user's transaction.

4. Rewriting (details: Appendix C)

Assignment 01 contained only two classes (`Logic`, `Database`), so we rewrote two **modules within them**:

1. **Budget calculation module** (`Logic.budget_status`, `Logic.set_budget`, the budget branches of `Logic.add_transaction`/`delete_transaction`, and `Database.get_single_budget`/`update_budget_remaining`/`get_budget_remaining_amount`). It became `BudgetCalculator`, a pure class whose remaining budget is *calculated* from transactions instead of a stored running balance. The stored balance was wrong for 2 of the 3 budgets in the real data. The new class removes the duplicated threshold logic and makes the add-expense alert agree with the dashboard.
2. **Expense management module** (`Logic.add_transaction` CC 16, `Logic.import_csv` CC 15, `Logic._validate_amount`/`_validate_date`, `Logic.delete_transaction`). It became the pure `ExpenseValidator` and `CsvImporter` plus a coordinating `ExpenseService`. CSV import is now atomic, short rows no longer crash, and one category-name rule applies to expenses and budgets.

Five defects were confirmed by running the baseline and fixed (`evidence/defect_probes.md`).

5. Testing and regression

All tests were executed. **225 passed, 0 failed** (`evidence/test_results.txt`). Coverage of `app/backend` is 97%.

Suite	Tests	Database	What it shows
`tests/unit`	106	none (mocks)	Rewritten classes; expense, budget, user and currency services with mocked repositories; repositories with a fake cursor; connection manager with mocked psycopg; controllers with mocked services
`tests/integration`	34	PostgreSQL test DB	The 5 required flows (store expense, retrieve, budget + summary, remaining after add/delete, category summary), sample CSV import, atomic import on DB failure, invalid inputs → 400, cross-user delete → 404, DB down → 503, constraints, migration script end to end
`tests/regression`	85	PostgreSQL test DB	All 73 Assignment 01 tests re-run on the new backend, plus 12 feature tests (add/view expenses, income, budgets, category summaries, chart data, CSV import/export, currency, frontend assets)

Tests use only `TEST_DATABASE_URL`, empty the tables before each test, and refuse to run against a database whose name lacks "test" or equals `DATABASE_URL`. Without a test database, the 119 DB tests are *skipped*, not failed.

All 73 original tests pass. Their requests, inputs and expected values were kept. What changed: the fixtures (a SQLite temp file became the PostgreSQL test DB), the call targets (old `Logic`/`Database` method names are mapped onto the new services and repositories), and the expected exception type for a duplicate username (`sqlite3.IntegrityError` became `psycopg.errors.UniqueViolation`). In addition, **six tests contain a deliberate change, each marked `A02`** in the code:

- the "invalid amount" test previously accepted an HTTP 500 and now requires 400;

- an unknown user now raises `NotFoundError`;
- the `Logic._convert` alias was removed;
- the schema check reads `schema_migrations`;
- deleting a transaction now requires the owner id;
- the stored `remaining_amount` column is no longer written.

The real frontend was also driven with Playwright against a migrated copy of the data: register, budget, two expenses (critical alert pop-up at 96%), income, sample CSV import (15/15) and month switch, with **no JS errors**. Screenshots are in `evidence/screenshots/`.

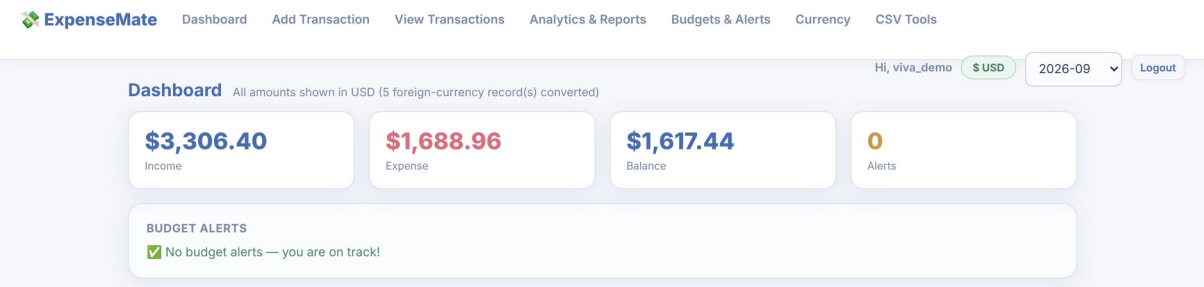


Figure 1: Dashboard after the browser test, September 2026, data imported from the sample CSV (PostgreSQL backend)

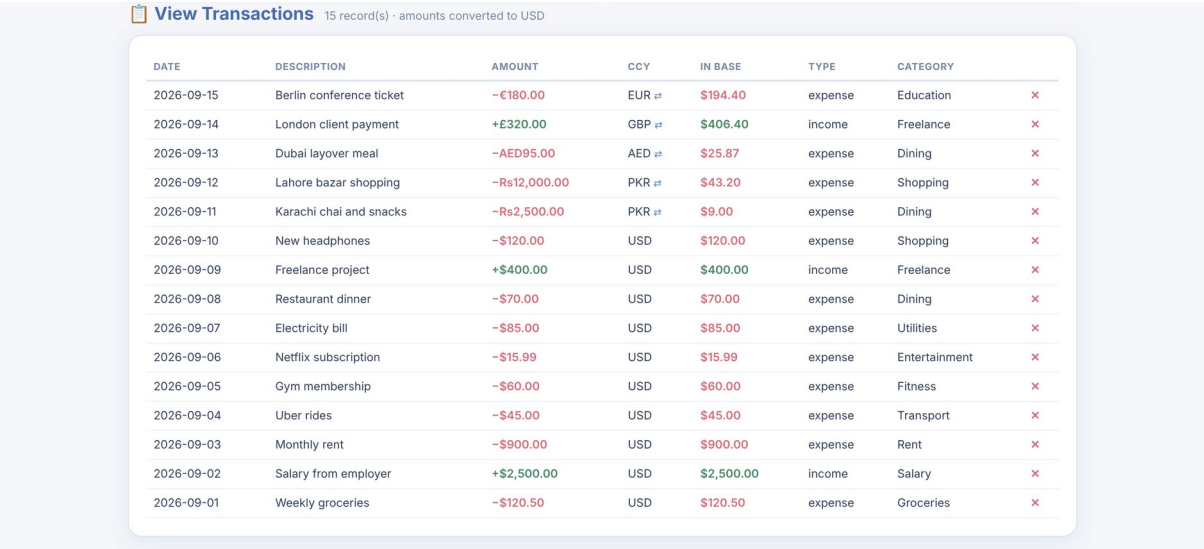


Figure 2: Transactions table with native and converted amounts, unchanged Assignment 01 frontend

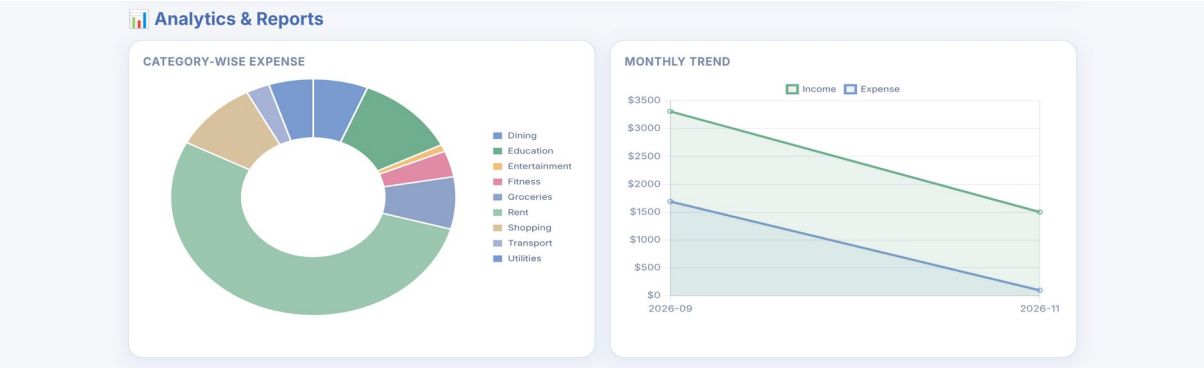


Figure 3: Category-wise expense and monthly trend charts (Chart.js) served from PostgreSQL data

CI (GitHub Actions) now starts a PostgreSQL service. It has **not been run on GitHub yet**, because the branch has not been pushed.

6. Maintainability metrics (details: Appendix E)

Indicator (radon / grep / pytest)	Before	After
Highest method CC	16	9
Methods with CC ≥ 6	4	1
Lowest file maintainability index	35.6	60.4
Largest class (methods)	26	8
Direct data-access calls from route handlers	4	0
SQL outside repository/database layer	0	0 (unchanged)
Tests / backend coverage	73 / 93%	225 / 97%
SLOC of backend	772	1129 (worse)

7. Team contributions

Member	Area of ownership (agreed division)
24K-3010 Erfa Kashif	PostgreSQL migration and script, database configuration (`database/`), repositories, `.env.example`, database/repository/migration tests, migration report
24K-3037 Daneya Shafiq	Layered architecture integration, services and controllers, rewriting of the budget and expense modules, frontend compatibility checks, regression tests, refactoring/rewriting/final reports
Both	Test review, regression run, metrics review, viva preparation

Accuracy note: the code, tests and reports in this branch were produced with an AI coding assistant (Claude Code) working on the team's repository, following the division above. Before submission, each member should review the commits in their area, re-run the tests, and edit this table if the real division of work differed. The commits currently carry the assistant's Git identity, not the team members' identities.

8. Challenges, trade-offs and lessons learned

- **Behaviour preservation vs fixing bugs.** We kept the API shapes byte-compatible so the frontend did not need to change, but deliberately changed buggy behaviour (500 → 400, cross-user delete → 404). Each change is listed in the refactoring report and pinned by a test.
- **Stored vs calculated values.** Calculating "remaining" re-reads the month's transactions on every request. That is slightly more work for the database, but there is one source of truth. At personal-finance scale this is negligible, and the `(user_id, date)` index supports it.
- **Legacy data.** We migrated data exactly, including inconsistencies (`'food'` and `'Food'`, a username with a trailing space, drifted balances), rather than "cleaning" it silently. Cleaning would be a separate, user-approved step.
- **Floats vs Decimal.** The database stores exact `NUMERIC` values, but services still calculate with `float`, to keep Assignment 01's results identical. A full switch to `Decimal` arithmetic is future work.

- **More files.** SLOC grew by 46%. Smaller units are easier to test and change, but newcomers need the architecture diagram.
- **Lesson:** measure before changing. Running the baseline exposed five real defects and the corrupted balances. Without that evidence, the rewrite would have been a matter of taste.

Appendix A: Migration Report

Project: ExpenseMate · **Course:** Software Construction & Development (SE-5B) · **Assignment 02**
Owner: 24K-3010 Erfa Kashif · **Script:** `migration/migrate_sqlite_to_postgres.py`

1. Before state (Assignment 01, SQLite)

- Database: one file, `app/data/expensemate.db`, accessed with Python's built-in `sqlite3` module.
- Schema: created at start-up by `Database.init_schema()` in `app/backend/db.py`. Five tables: `users`, `categories`, `transactions`, `budgets`, `settings`.
- Types: money was stored as `REAL` (binary floating point), and dates and timestamps as `TEXT`. Foreign keys were only enforced because every connection ran `PRAGMA foreign_keys = ON`.
- Connections: every one of the 21 `Database` methods opened and committed its own connection. A service could not group several writes into one transaction. For example, "insert expense" and "update the budget balance" committed separately.
- Configuration: the file path was hard-coded, with an optional `EXPENSEMATE_DB` override.
- Data at migration time: users = 2, categories = 11, transactions = 12, budgets = 3, settings = 0.

2. Target state (Assignment 02, PostgreSQL 16)

- Driver: **psycopg 3** (`psycopg[binary]`), using `dict_row` rows and `%s` parameter placeholders.
- Configuration: `DATABASE_URL` (application) and `TEST_DATABASE_URL` (tests) are read from the environment or a git-ignored `.env` file. `.env.example` documents them and holds no real secrets.
- Schema: versioned SQL files in `app/backend/database/migrations/` (`001_initial_schema.sql`), applied by `apply_migrations()`. That function is idempotent, records each version in `schema_migrations`, and uses an advisory lock so several workers can start at the same time.
- Types: `NUMERIC(14,2)` for money, `DATE`/`TIMESTAMP` for time values, `INTEGER GENERATED BY DEFAULT AS IDENTITY` primary keys, all original foreign keys (including `ON DELETE CASCADE`), all `CHECK` and `UNIQUE` constraints. It also adds a `month` format check and an index on `transactions(user_id, date)`.
- Transactions: `Database.transaction()` shares one connection per thread, so a service can commit or roll back several repository calls as one unit.

3. Reason for migration

1. **Concurrency.** SQLite locks the whole file for each write. PostgreSQL handles many simultaneous users through row-level locking (MVCC).
2. **Correct money arithmetic.** `NUMERIC` stores exact cents. `REAL` cannot (`0.1 + 0.2 ≠ 0.3`).
3. **Real data types and enforced integrity.** Real `DATE` values, and constraints that are always on.

4. **Deployability.** A file inside the app folder disappears on serverless or container hosts (the project has `api/index.py` for Vercel). A managed PostgreSQL server persists independently of the app.
5. **Separation of credentials from code.** Connection details now come from environment variables.

4. Implementation procedure

1. Create the role and the databases (`expensemate`, `expensemate_test`). See `migration/README.md`.
2. `cp .env.example .env` and fill in the URLs.
3. `python migration/migrate_sqlite_to_postgres.py --report docs/evidence/migration_verification.md`

The script then:

1. opens SQLite **read-only** (`mode=ro`) and records its SHA-256 checksum;
2. runs `integrity_check` and `foreign_key_check`;
3. converts the types (see the table below);
4. creates the schema from the application's own migration files, so there is one source of truth;
5. in **one PostgreSQL transaction**, copies all rows with their original ids, resets the identity sequences, and verifies the copy;
6. **commits only if every check passes**, otherwise rolls back.

Compatibility issue	SQLite	PostgreSQL / how it was handled
Auto-increment keys	`AUTOINCREMENT`	`IDENTITY BY DEFAULT`. Original ids are inserted explicitly, then `setval(MAX(id)+1)`
Money	`REAL`	`NUMERIC(14,2)`. Values are converted via `Decimal(str(x))` and the script warns if any are rounded (none were)
Dates/timestamps	`TEXT`	`DATE`/`TIMESTAMP`. An invalid date **aborts** the migration
Month filter	`strftime('%Y-%m', date) = ?`	`date >= first_day AND date < next_month` (can use the index)
Placeholders / new id	`?`, `cursor.lastrowid`	`%s`, `INSERT … RETURNING id`
Upsert	`ON CONFLICT … excluded.x`	same syntax, kept
Row values returned	`str`/`float`	`Decimal`/`date`/`datetime` are converted back to the old API formats in `models/` (amounts as numbers, `YYYY-MM-DD`, `YYYY-MM-DD HH:MM:SS`), so the frontend is unchanged
Schema bug	`remaining_amount … CHECK (amount >= 0)` (the check tested the wrong column)	no check on the legacy column: a negative remaining value is legitimate
Case-sensitive data	`'erfa '` (trailing space), `'food'` and `'Food'`	copied byte-for-byte. Nothing is "cleaned" during migration

5. Data preservation

Every row of every table is copied with the same primary key, so all relationships are preserved. The original SQLite file is never written: it is opened read-only and its checksum is compared before and after (`d985437f…28a6` both times). The script refuses to overwrite a non-empty target unless `--replace` is given. The legacy `budgets.remaining_amount` values are preserved, even though the application no longer relies on them (see the rewriting report).

6. Risks and mitigation

Risk	Mitigation
Partial copy after an error	Single transaction with rollback. Tested: a corrupt row means 0 rows in PostgreSQL afterwards (`test_failure_rolls_back_everything`)
Precision loss	Exact `Decimal` conversion, rounding warnings, money totals compared per user/type/currency
Id collisions with new rows	Sequences reset and verified ("next id" checks); a new expense after migration got id 37 = max + 1 (tested)
Accidental double run / overwrite	Exits with code 2 ("refused") unless `--replace` is given
Credentials leaking	URLs only in `.env` (git-ignored); passwords masked in logs and reports
Damaging the source	Read-only open plus a checksum comparison
Tests wiping real data	Tests use only `TEST_DATABASE_URL`, and refuse to run if the name lacks "test" or equals `DATABASE_URL`

7. Verification evidence (actual run, `docs/evidence/migration_verification.md`)

Check	Result
Row counts: users 2/2, categories 11/11, transactions 12/12, budgets 3/3, settings 0/0	PASS
Row-by-row, column-by-column comparison (5 tables)	PASS: all rows identical
Money totals per user/type/currency (5 groups)	PASS
Orphan foreign keys	PASS: 0
Next generated id > max migrated id (4 tables)	PASS
SQLite checksum unchanged	PASS

Automated tests: `tests/integration/test_migration_script.py` has 6 tests covering a full copy, data readable by the app, a refused re-run, `--replace`, rollback on a corrupt row, an invalid date, and the precision warning. All pass. The application itself was run on the migrated data and driven in a browser (`docs/evidence/screenshots/`).

Appendix B: Refactoring Report

Owner: 24K-3037 Daneya Shafiq (layers, services, controllers) with 24K-3010 Erfa Kashif (repositories, database) **Commit:** `5b0e345 refactor: restructure backend into Controller -> Service -> Repository -> Database`

1. Old architecture

Assignment 01 had three backend modules: `server.py` (Flask routes), `logic.py` (`class Logic`) and `db.py` (`class Database`). The docstrings described them as layers, but the code did not respect that:

- `server.py` route handlers called the database directly in 4 places (`db.get_user_by_name`, `db.create_user` and `db.list_categories` in register/login/categories). They also called the *private* `logic._normalise()` in 3 places.
- `Logic` was a god class: 26 methods spanning currency conversion, users, validation, transactions, analytics, budgets/alerts and CSV.
- `Database` held SQL for all five tables plus the schema and connection handling. Each method committed alone.
- Route handlers had no error handling for invalid expenses or budgets, so bad input produced **HTTP 500** (verified on the baseline: `docs/evidence` probe results in the rewriting report).

Diagrams for both versions are in `architecture_before_after.md`.

2. New architecture: Controller → Service → Repository → Database

Layer	Package	Responsibility	Must not contain
Controller	`controllers/`	Read the request, basic checks (`json_body`, `require_fields`), call one service, return JSON; central error handlers	SQL, calculations
Service	`services/`	Business rules, calculations, coordination, transaction boundaries	SQL, Flask
Repository	`repositories/`	Parameterised SQL, row-to-model mapping	Business rules
Database	`database/`	Config from env, connections, `transaction()`, schema migrations	Business rules

These rules are checked mechanically: a grep shows no SQL keyword or `cursor()` call in `controllers/` or `services/`, and no `flask`/`psycopg` import in `services/`. `tests/unit/test_repositories.py` also asserts that repository modules contain no budget-rule constants.

3. The two refactored features

Feature 1: Add Expense. `ExpenseController.add_transaction → ExpenseService.add_expense → ExpenseRepository.add → PostgreSQL`. The controller checks required fields and the JWT user. The service validates, sets the default currency and category, and runs the category lookup and the insert in **one transaction**. The repository runs `INSERT … RETURNING id`.

Feature 2: Monthly Budget Summary. `BudgetController.summary → BudgetService.monthly_summary → BudgetRepository.list_for_month → PostgreSQL`. The controller validates `?month=`. The service gathers the spending (via `AnalyticsService`) and the budgets, converts currencies, and delegates the rules to `BudgetCalculator`. A new read-only endpoint, `GET /api/budgets?month=`, exposes just the budget status.

The JSON response shapes are unchanged, so `app/frontend` was **not modified**. This was confirmed by the 27 original API tests and a Playwright run of the real page with no JS errors.

4. Dependency direction and injection

`server.py:build_services()` builds the repositories with the `Database`, the services with the repositories, and the controllers (blueprint factories) with the services. Nothing creates its own dependencies. As a result, every unit test can pass in a `MagicMock` instead: controllers with mocked services, services with mocked repositories, and repositories with a fake cursor.

5. Code-quality improvements (with evidence)

1. **Single responsibility.** The largest class shrank from 26 methods (`Logic`) to 8 (`ExpenseService`). The worst cyclomatic complexity fell from 16 to 9 and the lowest file maintainability index rose from 35.6 to 60.4 (`maintainability_comparison.md`).
2. **No layer bypasses.** Direct data-access calls from route handlers went from 4 to 0, and calls to private service methods from controllers went from 3 to 0 (`CurrencyService.normalise` is now public).
3. **Consistent error handling.** One `register_error_handlers()` maps typed exceptions to 400/401/404/409/503 and logs real DB errors without exposing SQL. Invalid expenses and budgets now return 400 instead of 500.
4. **Correct authorisation.** Assignment 01 let any logged-in user delete *any* transaction by id, and missing ids returned "ok". The repository now deletes `WHERE id = %s AND user_id = %s` and the controller returns 404 (`test_users_cannot_delete_each_others_transactions`).
5. **Atomic operations.** `Database.transaction()` groups several repository calls. A CSV import is now all-or-nothing on DB failure (tested).
6. **Testability.** There are 106 unit tests that run in about 0.2 s with no database. The original suite could only run with a real DB file.
7. **Configuration.** Credentials and the JWT secret come from env/.env. Debug mode is off unless `FLASK_DEBUG=1`. Assignment 01 always ran with `debug=True`.

6. Behaviour intentionally changed (documented, all tested)

Situation	Assignment 01	Assignment 02
Invalid expense (e.g. amount −5)	HTTP 500	400 + message
Budget without category	HTTP 500	400
Delete someone else's / missing transaction	200, deleted / "ok"	404, nothing deleted
Month `2026-13`	accepted	400

Situation	Assignment 01	Assignment 02
Unknown user id in a service call	`ValidationError`	`NotFoundError` (404)
Unused code removed	`Database.delete_budget`, `get_budget_remaining_amount`, `Logic._convert`, `Logic.is_supported`	n/a (no callers)

Appendix C: Rewriting Report

Owner: 24K-3037 Daneya Shafiq · **Commits:** `67d627d rewrite: …`, `c47ac09 refactor(csv): …`

What existed

The Assignment 01 backend had only **two real classes**: `Logic` (`app/backend/logic.py`, 26 methods) and `Database` (`app/backend/db.py`, 21 methods), plus the exception `ValidationError`. There was no `ExpenseManager` or `BudgetCalculator` class. The expense and budget responsibilities were *modules inside those two classes*. We therefore rewrote the two responsibility clusters below and replaced them with new classes. We did not simply rename methods.

Rewrite 1: Budget calculation module

Original code (exact names):

- `Logic.budget_status`, `Logic.set_budget` (`app/backend/logic.py`)
- the budget branch of `Logic.add_transaction` (about 50 lines) and the credit-back branch of `Logic.delete_transaction`
- `Database.set_budget`, `Database.get_single_budget`, `Database.update_budget_remaining`, and the unused `Database.get_budget_remaining_amount` (`app/backend/db.py`)

Original responsibilities: store budgets, keep a *running balance* column `budgets.remaining_amount` (decreased on every expense insert, increased on every delete), compute the warning (80%) and critical (95%) levels, and format alert messages.

Problems found in the code and confirmed by running it (`docs/evidence/defect_probes.md`):

1. **Two sources of truth.** `budget_status` computed "remaining" from the transactions, while `add_transaction` used the stored `remaining_amount`. `set_budget` reset the stored value to the full amount even when money had already been spent. Result: after spending 91 of 100, the dashboard said *warning* but the add-expense alert said *nothing*. In the **real Assignment 01 data**, 2 of 3 stored balances were wrong. For example, Groceries showed 29,699.78 PKR remaining on a 10,000 PKR budget (`docs/evidence/legacy_remaining_vs_calculated.md`).
2. **Duplicated rules.** The threshold comparisons and both alert messages were written twice, in `add_transaction` and in `budget_status`.
3. **Mixed concerns and complex conditionals.** Data retrieval, currency conversion, persistence and the rules were interleaved. `add_transaction` had a cyclomatic complexity of 16, and a "guard against corrupted historical remaining amounts" hack showed the design was already fighting itself.
4. **Inconsistent category names.** Expenses were title-cased and budgets were not, so a budget for "groceries" never saw any spending.
5. **Hard to test.** Every rule needed a database and a full transaction insert to exercise.

Why a rewrite instead of a small refactor: the defect is in the *design*, not in a line of code. A stored running balance must be patched correctly on every insert, delete, budget change and currency

change, forever. Fixing one path, as Assignment 01 tried with its "FIX" comments, left the others wrong. Removing the stored balance and **calculating** it changes the algorithm, the data flow, the repository API (two methods deleted) and both callers. That is more than extracting a method.

New design: `services/budget_calculator.py`

- `BudgetThresholds` (frozen dataclass, validated: `0 < warning ≤ critical`, default 0.80/0.95)
- `BudgetCalculator.usage_ratio`, `level_for`, `evaluate`, `alert_for`, `evaluate_month`. These are **pure functions of numbers**, with no DB, HTTP or currency table. They return the `BudgetStatus`/`BudgetAlert` dataclasses (`models/budget.py`).
- `BudgetService` gathers the data (budgets and spending), converts currencies, and calls the calculator.
- `utils/validators.normalise_category()` applies one naming rule to both expenses and budgets.

Changes in dependent code: `ExpenseService.add_expense` asks `BudgetService.alert_for_category()`, which uses the same calculation as the dashboard. `delete_expense` no longer does any budget bookkeeping. `BudgetRepository.update_remaining`/`get_for_category` were deleted, and `upsert` no longer writes `remaining_amount` (migrated values are left untouched). `server.py` injects `BudgetCalculator()`.

Behaviour preserved: the same 80%/95% thresholds, the identical alert message text, the same `status` dict keys, and currency conversion into the base currency. All Assignment 01 budget tests still pass, including "remaining 100 → 60 after expense → 100 after delete". **Behaviour changed on purpose:** a zero budget with spending is reported as *critical* (it was "ok, 0%"), and the stored column is no longer maintained (the one legacy assertion on it is updated and marked `A02`).

Unit tests: `tests/unit/test_budget_calculator.py` has 20 tests (boundaries 0.7999/0.80/0.9499/0.95, custom and invalid thresholds, zero budgets, negative remaining, rounding, exact message texts, a whole month, no budgets). `tests/unit/test_budget_service.py` has 12 tests with mocks. Integration tests: `test_4_…` and `test_4b_…` (spending before the budget exists).

Rewrite 2: Expense management module

Original code (exact names): `Logic.add_transaction` (CC 16), `Logic.import_csv` (CC 15), `Logic._validate_amount`, `Logic._validate_date`, `Logic.delete_transaction` (`app/backend/logic.py`).

Original responsibilities: validate input, choose default currency and category, write the category and the transaction, update the budget, compute and format alerts, and parse CSV by calling `add_transaction` once per row.

Problems:

1. `add_transaction` did six jobs: validation, defaults, two DB writes, budget arithmetic and message formatting.
2. Validation was tangled with persistence. It could not be reused or tested without a database.

3. `import_csv` opened about 5 database connections **per CSV row**. A failure half-way left a **partial import**. A short row (`2026-09-01,only two`) crashed with `AttributeError` (`None.strip()`).
4. `_validate_amount(amount, type)` validated the *type* inside the amount check (hidden coupling) and accepted `NaN`/`inf`.
5. `delete_transaction` did not check ownership (any user could delete any id), and when the id did not exist it returned `False`, which the route ignored, so it answered `{"ok": true}`.

Why a rewrite: the method had no seams to extract along. Validation, persistence and budget logic shared local variables throughout. Making CSV import atomic also requires a different structure: validate everything first, then save in one transaction.

New design:

- `ExpenseValidator` (`utils/validators.py`) turns raw input into a validated, unsaved `Transaction`. It is pure; its only dependency is a currency-normalising function. It is reused for single entries and for CSV rows.
- `CsvImporter` (`services/csv_importer.py`) turns text into row dicts. It is pure, handles BOMs, header case and short rows, and counts blank rows. It was split again in `c47ac09` (CC 11 → 5).
- `ExpenseService` coordinates: `validate → transaction(get_or_create category, insert) → BudgetService alert`. `import_csv` validates all rows and then saves the valid ones **in one transaction** with a category cache. `delete_expense` is a single owner-scoped delete.

Behaviour preserved: the same validation messages, the same title-casing and "Income"/"Misc" defaults, the same `(id, alert)` and `(imported, skipped)` return values, and malformed CSV rows are still skipped rather than fatal. All 33 original `Logic` tests and 27 API tests pass.

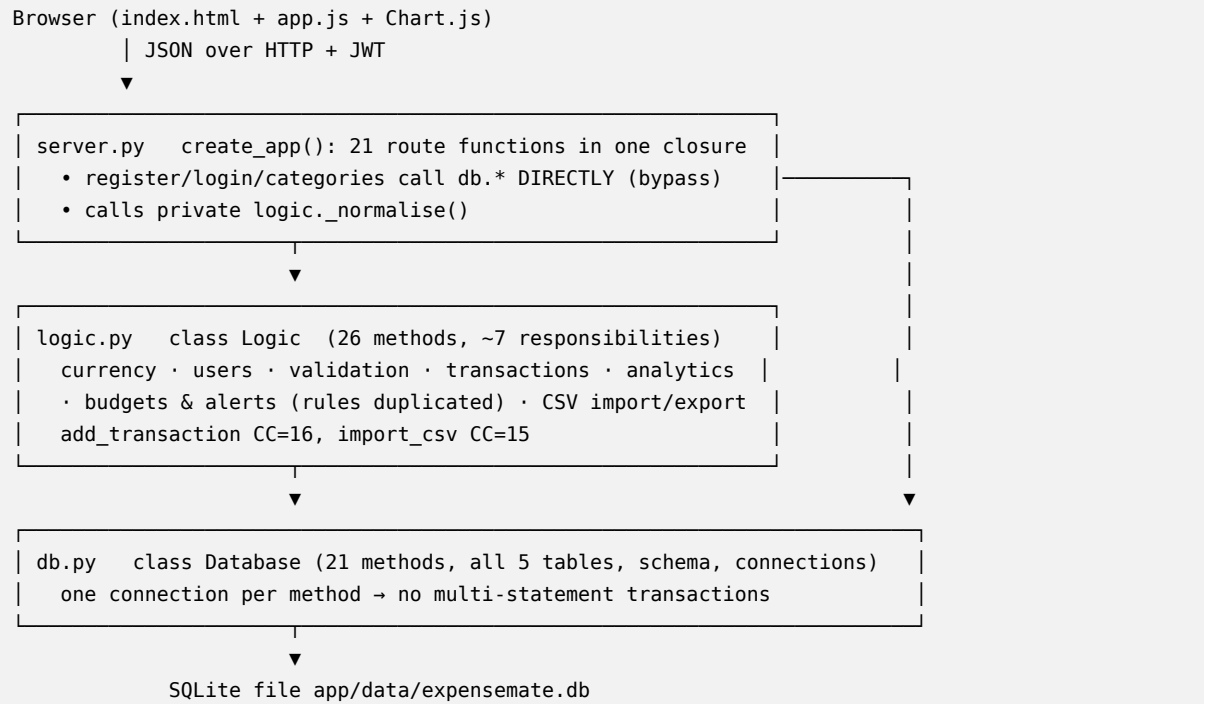
Unit tests: `test_expense_validator.py` (18), `test_csv_importer.py` (8) and `test_expense_service.py` (12, with mocks, e.g. "invalid expense never reaches the repository" and "import uses one transaction and two category lookups for five rows"). Integration: `test_csv_import_is_all_or_nothing_on_database_failure`.

Result

	Before	After
Worst method CC (radon)	16 (`Logic.add_transaction`), 15 (`import_csv`)	`add_expense` 2, `import_csv` 4, `CsvImporter.parse` 5, `BudgetCalculator.evaluate` 3
Places where alert thresholds are compared	2 (4 comparisons)	1 (`level_for`, 2 comparisons)
DB needed to test budget rules / validation	yes	no (pure classes)
Five confirmed defects	present	fixed and covered by tests

Appendix D: Architecture Before and After

Before (Assignment 01, tag `assignment-01-baseline`)



The modules were labelled "layers", but the layering was not enforced: the HTTP code called the database directly, and `Logic` mixed business rules with data coordination.

After (Assignment 02, branch `assignment-02`)



Dependency direction: every arrow points downwards. Controllers know only services. Services know repositories, other services and models. Repositories know only `database` and models. Nothing imports a higher layer. `server.py` is the single *composition root* that builds the objects and injects them through constructors. This is a layered architecture, not MVC: there are no view templates or "model controllers", and the frontend is a separate static client.

Feature flows

Feature 1: Add Expense (`POST /api/transactions`)

```
ExpenseController.add_transaction      require date/amount/type, read JWT user
  → ExpenseService.add_expense
      → UserService.get_user           default currency
      → ExpenseValidator.validate      pure: date, type, amount, currency, description, category name
      → Database.transaction():
          CategoryRepository.get_or_create   INSERT … ON CONFLICT DO NOTHING RETURNING id
          ExpenseRepository.add              INSERT … RETURNING id
      → BudgetService.alert_for_category → BudgetCalculator (same rules as the dashboard)
  ← 201 {"status":"success","id":…, "alert":…}
```

Feature 2: Monthly Budget Summary (`GET /api/summary?month=YYYY-MM`, also `GET /api/budgets`)

```
BudgetController.summary               validate month (default: current month)
  → BudgetService.monthly_summary
      → AnalyticsService.category_summary → ExpenseRepository.list_for_user (month range)
      → BudgetRepository.list_for_month   SELECT budgets JOIN categories
      → CurrencyService.convert           limits → base currency
      → BudgetCalculator.evaluate_month   remaining, ratio, level, alerts
      → AnalyticsService.monthly_trend
  ← 200 {totals, by_category, by_currency, budget:{status, alerts}, trend}
```

Folder structure (actual)

```
Expense-Mate/
├── app/
│   ├── backend/
│   │   ├── controllers/  __init__, _request, auth_, budget_, currency_, expense_controller, error_handlers
│   │   ├── services/     __init__, expense_service, budget_service, budget_calculator, csv_importer,
│   │   │                 analytics_service, currency_service, user_service
│   │   ├── repositories/ __init__, expense_repository, budget_repository, category_repository, user_repository
│   │   ├── database/     __init__, config, connection, schema, migrations/001_initial_schema.sql
│   │   ├── models/       __init__, expense, budget, category, user, _convert
│   │   ├── utils/        __init__, validators, errors, logger
│   │   ├── server.py
│   │   └── __init__.py
│   ├── frontend/         (unchanged) index.html, static/app.js, chart.umd.min.js, style.css
│   ├── data/expensemate.db   (original SQLite, kept read-only as migration source)
│   └── sample_data/sample_expenses.csv
├── tests/  conftest.py, unit/, integration/, regression/
├── migration/  migrate_sqlite_to_postgres.py, README.md
├── docs/   reports + evidence/
├── api/index.py   (serverless entry point)
├── requirements.txt · .env.example · .gitignore · README.md · run.py
```

The folders deviate slightly from the suggested template, for reasons tied to the real code. There are extra `analytics/currency/user` services and `auth/currency` controllers because Assignment

01 already had those features. There are `category`/`user` models and repositories because those tables exist. `budget_calculator.py` and `csv_importer.py` are the rewritten modules.

Appendix E: Maintainability Comparison

Before = tag `assignment-01-baseline` (commit `390ed18`). **After** = branch `assignment-02`. Both were measured with the same commands on the same machine, on 2026-11-07. The raw outputs are in `docs/evidence/metrics_before.txt`, `metrics_after.txt` and `test_results.txt`. The "before" numbers were re-measured from a git worktree of the tag, so anyone can reproduce them.

How each metric was measured

Metric	Tool / command (run inside `app/backend/`)
Cyclomatic complexity (CC)	`radon cc -s -a -n A .` (average) and `radon cc -s -n B .` (blocks with CC ≥ 6)
Maintainability index (MI)	`radon mi -s .` (0–100, higher is better)
Source lines	`radon raw -s .` (SLOC total)
Largest class	`ast` script counting `def` statements directly inside each class (in the evidence files)
SQL location	`grep -E 'SELECT … FROM|INSERT INTO|UPDATE x SET|DELETE FROM'` per file
Layer bypasses	route handlers calling `db.*` (before) or any repository/cursor (after) in controllers; calls to `logic._private` methods
Duplicated rules	number of `>= CRITICAL_RATIO / WARN_RATIO` (before) vs `>= self.thresholds.*` (after) comparisons
Tests and coverage	`pytest --cov=app/backend`

Results

#	Indicator	Before	After	Change
1	Highest CC of any method	**16** (`Logic.add_transaction`), 15 (`Logic.import_csv`)	**9** (`AnalyticsService.category_summary`)	better
2	Methods with CC ≥ 6 (radon rank B or worse)	4 (16, 15, 10, 10)	1 (9)	better
3	Average CC per block	2.87 (53 blocks)	2.31 (137 blocks)	better
4	Lowest file MI	35.55 (`logic.py`)	60.35 (`expense_repository.py`)	better
5	Largest class (methods)	26 (`Logic`), 21 (`Database`)	8 (`ExpenseService`)	better
6	Responsibilities in the largest class	~7 in `Logic`: currency, users, validation, transactions, analytics, budgets/alerts, CSV	1 per class (e.g. `BudgetCalculator` = budget rules only)	better (qualitative count from reading the methods)
7	Route handlers calling data access directly	4	0	better

#	Indicator	Before	After	Change
8	Controller calls to private service methods	3 (`logic._normalise`)	0	better
9	SQL statements outside the repository/database layer	0 (all 22 in `db.py`)	0 (16 in 4 repositories + 3 in `database/schema.py`)	**unchanged**: already true before
10	Budget-threshold comparisons (duplication)	4, in 2 places	2, in 1 place (`BudgetCalculator.level_for`)	better
11	Automated tests	73	225 (106 unit, 34 integration, 85 regression)	more
12	Tests runnable without a database	0	106 (≈0.2 s)	better
13	Statement coverage of `app/backend`	93% of 542 statements	97% of 890 statements	better
14	Total SLOC of `app/backend`	772 (4 files)	1129 (36 files)	**worse: +46%**

Interpretation (honest)

- The clearest gains are in **complexity concentration** (indicators 1, 2, 4 and 5) and **dependency separation** (7 and 8). No single unit is now hard to read or test. The two most complex Assignment 01 methods, CC 16 and 15, were exactly the two rewritten modules.
- **Indicator 9 did not improve.** Assignment 01 already kept SQL inside `db.py`. The gain is that the SQL is now split per table, and services group statements into transactions, which `db.py` could not do.
- **The code base is larger** (indicator 14). This is the cost of more files, docstrings, dataclass models, explicit dependency injection and the migration tooling. More, smaller units are easier to change locally, but there is more to navigate. A student needs the architecture diagram to find things.
- Coverage numbers compare different amounts of code (542 vs 890 statements), so they show that the new code is at least as well tested, not that testing "improved by 4%".
- `AnalyticsService.category_summary` (CC 9) is now the most complex method. It is a candidate for further splitting, but it was only moved (not rewritten) and all its tests pass.

Appendix F: Change Log

Baseline: tag **`assignment-01-baseline`** (and branch `assignment-01`) → commit `390ed18` (2026-11-05). Neither is modified. All Assignment 02 work is on branch **`assignment-02`**.

"Responsible" shows the team member who **owns** that area for review and the viva, following the agreed division. The implementation in this branch was produced with the help of an AI coding assistant (Claude Code). Each owner must review their commits and be able to explain them (see "Team contributions" in `final_report.md`).

Version / commit	Date	Files / classes affected	Change performed	Reason	Responsible
`390ed18` (tag `assignment-01-baseline`)	2026-11-05	whole project	Assignment 01 final state (SQLite)	Preserved baseline	Erfa Kashif, Daneya Shafiq
`6093473`	2026-11-07	`.gitignore`, `app/backend/__pycache__/*`, `tests/__pycache__/*`	Stopped tracking compiled `.pyc` files; ignore `.env`, coverage output, virtualenvs	Bytecode changed on every test run; keep secrets out of Git	Erfa Kashif
`84e03b2`	2026-11-07	`app/backend/database/{config,connection,schema}.py`, `migrations/001_initial_schema.sql`, `utils/logger.py`, `migration/migrate_sqlite_to_postgres.py`, `migration/README.md`, `.env.example`, `requirements.txt`, `docs/evidence/migration_verification.md`	PostgreSQL configuration from env, connection + transaction manager, versioned schema, verified migration script (16/16 checks passed on real data); added psycopg, removed unused matplotlib	Part A: SQLite → PostgreSQL	Erfa Kashif
`5b0e345`	2026-11-07	**removed** `db.py` (`Database`), `logic.py` (`Logic`); **new** `controllers/*`, `services/{currency,user,analytics,expense,budget}_service.py`, `repositories/*`, `models/*`, `utils/{errors,validators}.py`; rewritten `server.py`, `run.py`, `api/index.py`; tests moved to `tests/regression/`	Controller → Service → Repository → Database; dependency injection; central error handling (400/404/409/503); owner-scoped delete	Part B: layered architecture; fix HTTP 500s and cross-user delete	Daneya Shafiq (controllers, services), Erfa Kashif (repositories)
`eb51cab`	2026-11-07	`sample_data/` → `app/sample_data/`	Moved folder	Part D target structure	Daneya Shafiq

Version / commit	Date	Files / classes affected	Change performed	Reason	Responsible
67d627d	2026-11-07	**new** services/budget_calculator.py (BudgetCalculator, BudgetThresholds), services/csv_importer.py (CsvImporter), utils/validators.py (ExpenseValidator, normalise_category), models/budget.py (BudgetStatus, BudgetAlert); rewritten ExpenseService, BudgetService; BudgetRepository (2 methods removed)	Remaining budget calculated instead of stored; one rule set for alerts; pure validator/CSV parser; atomic CSV import; one category-name rule	Part C: rewrite budget-calculation and expense-management modules (5 confirmed defects)	Daneya Shafiq
8c8c277	2026-11-07	tests/unit/* (8 files), tests/integration/* (3 files), tests/regression/test_assignment01_features.py, tests/conftest.py, .github/workflows/python-tests.yml	225 tests on a separate PostgreSQL test DB with safety guard; CI with a postgres:16 service	Part E: testing	Erfa Kashif (DB/repository/migration tests), Daneya Shafiq (service/controller/regression tests)
c47ac09	2026-11-07	services/csv_importer.py; docs/evidence/*	Split CsvImporter.parse (CC 11 → 5); added metrics, screenshots, data-drift evidence	Part F finding; evidence	Daneya Shafiq
165394f	2026-11-07	docs/*.md, README.md	Migration, refactoring, rewriting, architecture, maintainability and final reports; README	Part H	Daneya Shafiq (reports), Erfa Kashif (migration report)
Word report commit	2026-11-07	docs/ExpenseMate_Assignment02_Report.docx, docs/maintainability_comparison.md, docs/change_log.md	Combined Word report (cover, contents, final report, appendices A–H, screenshots); fixed a Markdown table that split on |	Submission document	Daneya Shafiq

Appendix G: Migration Verification Output

Run at: 2026-11-07T12:32:11

- Source (SQLite, read-only): `app/data/expensemate.db`
- Target (PostgreSQL): `postgresql://expensemate:****@localhost:5432/expensemate`
- SQLite integrity_check: ok; foreign_key_check: no violations
- Rows read: users=2, categories=11, transactions=12, budgets=3, settings=0
- PostgreSQL schema migrations applied now: ['001']

Check	Result	Detail
row count: users	PASS	SQLite=2 PostgreSQL=2
row count: categories	PASS	SQLite=11 PostgreSQL=11
row count: transactions	PASS	SQLite=12 PostgreSQL=12
row count: budgets	PASS	SQLite=3 PostgreSQL=3
row count: settings	PASS	SQLite=0 PostgreSQL=0
row-by-row: users	PASS	all rows identical
row-by-row: categories	PASS	all rows identical
row-by-row: transactions	PASS	all rows identical
row-by-row: budgets	PASS	all rows identical
row-by-row: settings	PASS	all rows identical
money totals per user/type/currency	PASS	5 groups compared
orphan foreign keys	PASS	0 orphan rows
next id: users	PASS	max id=2, next generated id=3
next id: categories	PASS	max id=11, next generated id=12
next id: transactions	PASS	max id=36, next generated id=37
next id: budgets	PASS	max id=8, next generated id=9

- SQLite SHA-256 before: `d985437fcb95340084ad4354a9151eb8775f7b876ae87b800c5e4628162528a6`
- SQLite SHA-256 after: `d985437fcb95340084ad4354a9151eb8775f7b876ae87b800c5e4628162528a6`
- Original SQLite file unchanged.

RESULT: MIGRATION COMMITTED - all verification checks passed.

Appendix H: Defect Probes (Before vs After)

Same five scenarios run against the code at tag assignment-01-baseline (SQLite) and at branch assignment-02 (PostgreSQL). The automated equivalents are in tests/integration/test_system_flows.py and tests/unit/test_budget_calculator.py.

Before (assignment-01-baseline)

```
1. invalid amount on POST /api/transactions -> HTTP 500
2. bob deletes alice's transaction -> HTTP 200 | alice rows left: 0
3. budget without category -> HTTP 500
4. after 91 spent of 100: budget_status level = warning remaining = 9.0 | add_transaction alert = None | stored
remaining_amount = 99.0
5. budget set as 'groceries', expense entered as 'groceries' -> budget rows: {'Food': 91.0, 'groceries': 0.0}
```

After (assignment-02)

```
1. invalid amount on POST /api/transactions -> HTTP 400 {'error': 'Amount cannot be negative.'}
2. bob deletes alice's transaction -> HTTP 404 | alice rows left: 1
3. budget without category -> HTTP 400 {'error': 'Missing required field(s): category'}
4. after 91 spent of 100: budget_status level = warning remaining = 9.0 | add_expense alert = {'category': 'Food', 'level':
'warning', 'message': "Budget for 'Food' is at 91% ($91.0 of $100.0)."}
5. budget 'groceries' + expense 'groceries' -> budget rows: {'Food': 91.0, 'Groceries': 45.0}
```